

ASearcher: dataset inspection report

Wave-0 M7 (rung 3, long-horizon web search), CA-Benchmark project

2026-07-25

Scope. Direct inspection of the ASearcher datasets (inclusionAI) for rung 3 of the credit-assignment benchmark, answering the three questions posed: **how many turns**, **whether expert trajectories exist**, and **whether there is a train/eval split**. All counts come from downloading and parsing the raw files (the HF dataset viewer is broken for these repos); reproduce with `scripts/asearcher/inspect_dataset.py`, assets in `assets/`. Paper facts are from arXiv 2508.07976 (verified).

1 Headline answers

1. **Turns: none in the data; horizon is a training-time setting.** ASearcher’s data is (question, answer) pairs with *no* interaction traces, so it carries no turn counts. The paper sets the RL *turn budget* to **32 turns** for the 7B/14B agents and **128 turns** for the QwQ-32B agent, with emergent behaviour exceeding **40 tool calls / 150k tokens** per trajectory. Our benchmark caps rung 3 at **30 turns** by design, so we sit at the 7B/14B operating point, not the 128-turn regime the “Beyond Ten Turns” title advertises.
2. **Expert trajectories: NO.** Neither training split contains any thought / search-query / retrieved-doc / turn field — verified by scanning every row’s schema. There is no published SFT / cold-start / demonstration dataset either. ASearcher trains RL (GRPO) *directly* from base models with no behaviour-cloning warm start. The closest thing to “expert” guidance anywhere is a human *reference-solution outline* attached to two *eval* sets (GAIA Annotator_Metadata.Steps, xbench reference_steps) — prose plans, not agent rollouts, and eval-only.
3. **Train/eval split: yes, as two separate repositories.** ASearcher-train-data holds two *training* splits (~70.6k QA pairs total, no held-out test inside it); evaluation is a distinct repo ASearcher-test-data with 10 benchmark subsets. Train and eval never live in one file.

Consequence for our plan. The proposal already anticipated this: rung 3’s SFT π_{base} must be *self-generated* (strong-model rollouts in our capped 30-turn setup, or DR-Tulu remap) because ASearcher ships none. And the solvability diagnostic (M7 step 4) is unavoidable — the data gives us questions, but whether *our* 4B agent under *our* wiki-18 corpus at 30 turns can answer them is exactly what the go/no-go gate must measure.

2 The training data: inclusionAI/ASearcher-train-data

Two JSONL files, both **QA pairs only**, Apache-2.0. Data is *synthesized*: a prompt-based LLM agent starts from 14,107 seed questions and applies “Injection” (add related facts) and “Fuzz” (obscure identifying details) operations, then a 3-step quality/difficulty/uniqueness filter. It is QA-pair synthesis, *not* trajectory synthesis.

Split	Rows	Fields
ASearcher-Base-35k	35,583	question, answer (list), source, aug_answer (aliases), qid
ASearcher-LRM-35k	35,054	question, answer (string), source, id, idx

The two splits are genuinely different products, not a random partition:

- **Base-35k** — shorter multi-hop questions (median **23** words, p95 60), a list-valued answer, and *heavy alias augmentation* (aug_answer: median 4 paraphrases, max 1454). Sources: compose_chain_qa 8,537, open-source-fm8k 7,991, open-source 7,927, compose_group_qa 6,375, and compose_chain_qa_invalid **4,753**. Note that last tag: ~13% of Base is flagged *invalid* yet shipped — we must decide to drop or keep it (drop, unless the diagnostic says otherwise).
- **LRM-35k** — longer, more convoluted “chain” questions (median **43** words, p95 88, max 639) aimed at large-reasoning-model agents; string answer, no aliases. Sources: 24,026 rows carry *no source tag* (bare numeric ids), chain_qa_fuzzy 3,527, math 2,930, train_v1:webwalker_hard **2,572**, train_v1:compose_group_qa 1,999.

The question-length gap (23 vs 43 median words) is itself a within-rung difficulty axis. The `math` slice in LRM and the missing aliases mean LRM is the harder, noisier half — relevant to which split we draw the frozen training set from.

3 The eval data: inclusionAI/ASearcher-test-data

A separate repo, one **test** split per benchmark subdirectory, all QA pairs (+ metadata), no trajectories. Row counts verified by download:

Subset	Rows	Notes
GAIA	103	has human <code>Annotator_Metadata.Steps</code> (reference plan); Level 1–3
frames (FRAMES)	824	multi-constraint; gold Wikipedia link chains + <code>reasoning_types</code>
xbench-deepsearch	100	Chinese deep-search; has <code>reference_steps</code> ; canary-guarded
Bamboogle	125	short multi-hop QA
2WikiMultihopQA_rand1000	1,000	random-1000 subsample
HotpotQA_rand1000	1,000	random-1000 subsample
Musique_rand1000	1,000	random-1000 subsample
NQ_rand1000	1,000	random-1000 subsample
PopQA_rand1000	1,000	random-1000 subsample
TriviaQA_rand1000	1,000	random-1000 subsample

Total 7,277 eval items across 10 benchmarks. The paper’s headline (web setting, ASearcher-Web-QwQ) reports GAIA ~53–59, xbench ~42–51, FRAMES ~71–75 (exact numbers are checkpoint-dependent between the arXiv version and the repo leaderboard — pin the checkpoint before citing).

4 Evaluation metric (and the training reward — they differ)

ASearcher uses **two different scoring functions** depending on whether it is training or evaluating; this matters because our proposal’s “EM, verifiable” for rung 3 matches only the *training* one.

Eval metric = LLM-as-Judge (“MBE”, the headline reported numbers). From `evaluation/evaluate.py`: the predicted answer and the gold answer are handed to a judge LLM with a binary-equivalence prompt — “*determine if the predicted answer is equivalent to the labeled answer ... respond Correct / Incorrect*” — returning a rationale plus a judgement; MBE = 1 iff judgement=correct, averaged over the set.

- **Judge model:** Qwen2.5-72B-Instruct (self-hosted default), or gpt-4o-mini via the OpenAI path.
- **Aggregation:** reported as **Avg@k** (mean correctness over k sampled rollouts) and **Pass@k** (correct if any of k), eval temperature 0.6, $k = \text{--pass-at-k}$ (docs advise raising k for stability). This judge metric is what they use on the free-form sets (GAIA, xbench, FRAMES) where exact match under-counts.

Training reward = rule-based, verifiable, no judge in the loop (`ASearcher/utils/rewards.py`): extract the last `<answer>...</answer>`, then score by **EM** with SQuAD-style normalization (lowercase, strip articles, drop punctuation, whitespace-fix, plus `True`→`yes` mapping and Chinese-token handling); **sub-EM** (substring) and token-**F1** variants are also implemented. No extractable answer ⇒ reward 0. The paper stresses there is deliberately no external LLM in the agent/training loop.

For us: rung-3 RL reward = their training EM/sub-EM/F1 (we already have this from rung 1’s Search-R1 harness). For the transfer eval on GAIA/xbench/FRAMES we adopt the judge track — with our pinned gpt-oss-120b substituted for Qwen2.5-72B — which is exactly the uniform-judge track in the harness (§6).

Caveat: any ASearcher SOTA number we cite must record *which judge* and *which k*, or it is not comparable (different judge model → different MBE).

5 Tool-calling interface (and what changes offline)

The **action grammar is identical online and offline** — only the backend behind each tool changes. The full ASearcher agent (`agent/asearcher.py`, `asearcher_reasoning.py`) emits three tags, generation stopping at each closing tag (`</search>`, `</access>`, `</answer>`):

Tool tag	meaning	backend: web setting → local (offline) setting
<code><search>query</search></code>	retrieval	Serper API → local dense-retriever <code>/retrieve</code> (e.g. e5 over wiki-18)
<code><access>url</access></code>	browse a page	Jina live extraction → local <code>/access</code> over a <i>prebuilt</i> url→page store
<code><answer>...</answer></code>	terminate	(returns final answer for scoring)

Does offline call the same tools? Yes — both `<search>` and `<access>` are supported offline. The bundled `tools/local_retrieval_server.py` exposes *both* a POST `/retrieve` (dense search over the corpus) and a POST `/access` endpoint. The crucial difference: `/access` is backed by a `PageAccess` class that serves pages from a *pre-built file keyed by URL* (Wikipedia pages, with `index.php/`→`index.php?title=` normalization) — so offline `<access>` can only open URLs that exist in that store, whereas online Jina fetches *any* URL live. `<search>` works over any corpus with a dense index (our existing wiki-18/e5).

Tool-calling options (three agent variants ship in the repo):

- **asearcher** — 2 tools: `<search>` + `<access>` + `<answer>`. The design that produces the long-horizon behaviour (search then browse specific pages).
- **asearcher-reasoning** — same 2 tools wrapped with an explicit `<thought>` block (for reasoning models).
- **search_r1** (`agent/search_r1.py`) — **search-only:** `<search>` + `<answer>` (Search-R1 grammar), *no* `<access>`. This is the variant that matches our existing rung-1 infra.

Decision this forces for rung 3 (flag for the manifest). Our current retrieval stack is *search-only* (rung 1 serves `/retrieve` over wiki-18, no page store). Two options: **(a) search-only rung 3** using the `search_r1` agent — zero new infra, reuses rung 1 exactly, but drops the browse tool that is central to ASearcher’s “deep search” and shortens realized horizons; **(b) full 2-tool** — additionally build the wiki-18 url→page store and stand up `/access`, reproducing ASearcher-local faithfully at real extra W0 cost. This choice interacts with the horizon claim (RQ2): without `<access>`, rung 3’s practical turn count and its separation from rung 1 both shrink, so **(b) is the more faithful long-horizon rung** if the budget allows. Recommend deciding at the M7 gate alongside the solvability diagnostic, since answerability under search-only vs search+browse can differ sharply.

6 Two leakage vectors we must handle (important)

1. **ASearcher train $\cap$ WebWalkerQA.** The LRM split contains **2,572 webwalker_hard items**. WebWalkerQA is one of *our* rung-3 transfer-eval benchmarks (proposal §4). Training on webwalker-derived questions and then reporting WebWalkerQA transfer would be leakage — the proposal’s “fuzzy dedup vs WebWalkerQA” step is therefore load-bearing, and may require dropping the whole `webwalker_hard` slice, not just exact-match hits.
2. **ASearcher eval $\cap$ our rung-1 SearchQA.** `ASearcher-test-data` is HotpotQA / 2Wiki / Musique / NQ / TriviaQA / PopQA / Bamboogle — the same families we *train on and eval* at rung 1. We must not let rung-3 training questions (composed from these) overlap rung-1 eval, and we should use *our own* rung-1 eval splits, not ASearcher’s rand1000 subsamples, to keep cross-rung comparisons clean. (Spot check: 0/125 exact-normalized overlap between ASearcher-train and the Bamboogle test file — reassuring for exact matches, but the composed questions demand fuzzy dedup, not exact.)

7 Environment / answerability note

ASearcher ships *two* settings: a **local-RAG** setting (Wikipedia-2018 corpus — the same wiki-18 dump we already host for rung 1) for its Local models, and a **live-web** setting (Serper + Jina) for its Web models. Its strongest published numbers are the *live-web* QwQ-32B agent. We run the **local-RAG, 30-turn, Qwen3-4B** configuration — strictly harder on answerability than live web and much smaller than QwQ-32B. **This is precisely why the M7 solvability diagnostic gates the rung:** the questions were synthesized to be hard for a 7B–32B agent with strong retrieval; our answerable-fraction check on 200–300 items under wiki-18 decides regime (i)/(ii)/(iii) before we spend training budget.

8 Challenges (rung-3 specific)

- **No warm start.** No expert trajectories $\Rightarrow$ rung-3 SFT π_{base} must be generated by us (strong-model rollouts in-setup, EM-verified), which is extra W0 inference cost and its own quality-control pass.
- **Answerability is unknown and setting-dependent.** Data is tuned for live-web / large models; under our capped local-RAG 4B agent the learnable-band fraction is the open question — the go/no-go gate.
- **Leakage on both sides** (webwalker into training, rand1000 benchmarks overlapping rung 1) — fuzzy dedup required, not exact.
- **Long, variable horizon** (paper 32–128 turn budgets; we cap 30) with retrieved-doc context that will not fit full-history — the rung-3 truncation policy (per-doc caps + drop-oldest `<information>`) is binding here, unlike the QA rung.
- **Noisy/heterogeneous training pool** — a 13% “invalid” slice in Base, a 40% source-less slice and a `math` slice in LRM; the frozen training draw needs an explicit inclusion policy.

Reproduce

`python scripts/asearcher/inspect_dataset.py --out <dir>` downloads both repos (public, no token) and regenerates `train_stats.json`, `test_stats.json`, `sample_rows.json`, `summary.md`.